GATE DA 2027

Programming, Data Structures and Algorithms

Full Edition: explanations, traces, solved problems

Python, core data structures, searching and sorting, divide-and-conquer and basic graph algorithms with the stories of Ramu and Somu

Ramu and Somu once more.
Ramu asks "but why?" about everything. **Somu** explains with small everyday stories: library shelves, queues at a canteen, a phone contact list, a town's roads.

Syllabus covered (GATE DA, Section 4): Python programming; core data structures (stacks, queues, linked lists, trees, hash tables); searching (linear and binary); sorting (selection, bubble, insertion, merge, quick); divide-and-conquer; introduction to graph theory; basic graph algorithms (traversals and shortest paths).

Every code example here was actually run and the outputs shown are the real outputs.

Contents

1 Python Programming **3**

1.1 Basic types and operators . . . 3
1.2 Mutable vs immutable objects . . . 4
1.3 Strings, lists, slicing . . . 5
1.4 Aliasing, copies and the nested-list trap . . . 5
1.5 Control flow, comprehensions, built-ins . . . 5
1.6 Functions, scope and recursion . . . 6
1.7 Classes and exceptions . . . 7
1.8 Strings: useful methods and formatting . . . 7
1.9 Output-tracing workshop (solved step by step) . . . 8

2 Asymptotic Analysis **10**

2.1 Counting loop iterations . . . 10

3 Core Data Structures **11**

3.1 Overview of operations . . . 11
3.2 Stack, queue, linked list . . . 11

3.3 Linked list operations in code . . . 12
3.4 Stack applications . . . 12
3.5 Queue implementations . . . 13
3.6 Hash tables . . . 14
3.7 Binary trees and binary search trees . . . 14
3.8 BST operations: search, insert, delete . . . 15
3.9 Rebuilding a tree from two traversals . . . 16
3.10 AVL trees and rotations . . . 16
3.11 Binary heap and priority queue . . . 17
3.12 Heapify and heap sort in code . . . 17

4 Searching **19**
4.1 Binary search variants and probe counts . . . 19

5 Sorting **21**
5.1 Summary table . . . 21
5.2 Insertion sort . . . 21
5.3 Merge sort . . . 22
5.4 Quick sort and partition . . . 22
5.5 Elementary sorts traced, and a sort-selection guide . . . 23

6 Divide-and-Conquer **25**
6.1 The recurrence and the Master theorem . . . 25
6.2 Classic divide-and-conquer algorithms . . . 26
6.3 Solving recurrences by substitution; more divide-and-conquer . . . 26

7 Introduction to Graph Theory and Basic Graph Algorithms **28**
7.1 Graph vocabulary and counting facts . . . 28
7.2 Breadth-first search (BFS) . . . 28
7.3 Depth-first search (DFS) . . . 29
7.4 Shortest paths . . . 30
7.5 Bonus: minimum spanning trees . . . 31
7.6 DFS timestamps, edge types and graph tests . . . 32
7.7 Bellman–Ford, Floyd–Warshall and Prim . . . 33

8 Solved GATE-style Problems **33**

9 One-page Cheat Sheet **35**

10 Practice Questions (with answers) **35**

1 Python Programming

Ramu & Somu: The calculator that does not lie

Ramu is learning to program and tries `-7 // 2` on Python. He expects -3 ("three and a half, drop the half"). Python says -4. Somu explains: `//` is *floor* division: it rounds towards $-\infty$, not towards zero. And Python's `%` then has the *sign of the divisor*, so that $a = (a//b)\cdot b + (a\%b)$ always holds: $-7 = (-4)(2) + 1$. GATE loves such details because they are easy to get wrong when you are in a hurry.

1.1 Basic types and operators

Type	Example	Mutable?	Notes
`int`	`42`	no	unlimited precision
`float`	`2.5`	no	64-bit; `round(2.5)` is `2` (round half to even)
`bool`	`True`	no	subclass of int; `True+True` is `2`
`str`	`"GATE"`	no	immutable; slicing makes a new string
`tuple`	`(1,2)`, `(1,)`	no	single element needs a trailing comma
`list`	`[1,2,3]`	**yes**	ordered, any items
`set`	`{1,2}`	yes	unique items, no order, hashable items only
`dict`	`{'a':1}`	yes	key → value; keys must be hashable

Operators you must be fluent with

- `/` true division (always `float`); `//` floor division; `%` modulo; `**` power (right associative: `2**3**2` is $2^9 = 512$).
- `7//2`= 3, `-7//2`= -4, `7%3`= 1, `-7%3`= 2, `5%-3`= -1, `divmod(17,5)`= $(3, 2)$.
- Comparison chains work as in maths: `3>2>1` is `True`, `3>2>2` is `False`.
- `==` compares values; `is` compares identity (same object). `[1]==[1]` is `True` but `[1] is [1]` is `False`.
- **Truthiness:** empty things and zero are false: `bool([])`, `bool(0)` are `False`; `bool("0")` and `bool(" ")` are `True` (non-empty strings).
- Logical operators short-circuit: in `a and b`, if `a` is false `b` is never evaluated; `x or y` returns the first truthy operand.

```
print(7//2, -7//2, 7%3, -7%3, 2**3**2, 10/4, 10//4)
print(bool([]), bool(0), bool("0"), bool(" "))
print([1]==[1], [1] is [1])
```

```
3 -4 1 2 512 2.5 2
False False True True
True False
```

1.2 Mutable vs immutable objects

Ramu & Somu: Whiteboard and printed photo

Ramu has a **whiteboard** and a **printed photo**. The whiteboard is *mutable*: erase a word, write another, and it is still the same whiteboard. The printed photo is *immutable*: to make it look different you must print a brand-new photo, and the old one stays exactly as it was. In Python, every value is such an object; the only question is whether it can be edited *in place*.

Definition and the two groups

Mutable = can be changed in place after creation: `list`, `dict`, `set`, most user-defined class objects.
Immutable = cannot be changed after creation (any "change" builds a new object): `int`, `float`, `bool`, `str`, `tuple`, `frozenset`, `None`.
Quick test: if an operation changes the object itself (`append`, `sort`, `pop`, `x[i]=...`) the type is mutable; if every operation returns a new value (`s.upper()`, `x+1`) it is immutable.

```
a = [1, 2, 3]; a[0] = 99; print(a) # list edited in place
s = "GATE"
try:
    s[0] = "X"
except TypeError as e:
    print("TypeError:", e)
t = "X" + s[1:] # builds a NEW string
print(s, t, id(s) == id(t))

l = [1, 2]; i1 = id(l); l.append(3); print(id(l) == i1) # same object
n = 5; i2 = id(n); n += 1; print(id(n) == i2) # new object
```

```
[99, 2, 3]
TypeError: 'str' object does not support item assignment
GATE XATE False
True
False
```

Why it matters (all of these are GATE favourites)

- **Aliasing:** `b = a` copies nothing. For a list, a change through `b` is visible through `a`; for an `int` or `str` no such effect is possible. (See the aliasing examples below.)
- **Function arguments:** a function can mutate a list argument in place (caller sees it); rebinding the parameter name (`lst = [9]`) does not affect the caller.
- **Mutable default argument:** `def f(x, L=[])` reuses one list for every call.
- **Hashing:** dictionary keys and set elements must be immutable (hashable). `{[1,2]: 'no'}` raises `TypeError: unhashable type: 'list'`; `{(1,2): 'ok'}` is fine.
- **A tuple holding a list:** `tp = (1, [2,3]); tp[1].append(4)` is allowed (gives `(1, [2, 3, 4])`) but `tp[0] = 9` raises `TypeError`. The tuple's *references* are fixed; the list it points to can still change, so such a tuple is not hashable either.
- **Methods returning `None`:** mutating methods (`list.sort()`, `append`) return `None`, so `x = x.sort()` destroys the list. `sorted(x)` returns a new list.

1.3 Strings, lists, slicing

Ramu & Somu: A row of lockers

Somu pictures a list as a row of numbered lockers starting at **0**. The last locker can also be called -1, the one before it -2, and so on. A **slice** `s[a:b:c]` means "start at locker a, stop *before* locker b, step by c". If you leave any part out, Python fills sensible defaults.

```
s = "GATE2027"
print(s[::-1], s[1:5], s[-3:], s[::2])
print(list(range(2,10,3)), list(range(5,0,-2)))
```

```
7202ETAG ATE2 027 GT22
[2, 5, 8] [5, 3, 1]
```

Slicing and `range`

- `s[a:b]` has length $\max(0, b-a)$ for step 1; indices past the end are clipped, so `"abc"[5:]` is `""` (no error), but `"abc"[5]` raises `IndexError`.
- `range(a,b,c)` gives $a, a+c, \ldots$ while $< b$ (or $> b$ if $c < 0$); `range(n)` has exactly n items.
- Strings and tuples are immutable: `s[0]='x'` raises `TypeError`. Lists are mutable.
- `a*2` on a list repeats it; `a+b` concatenates into a *new* list; `a+=b` extends `a` *in place*.

1.4 Aliasing, copies and the nested-list trap

Ramu & Somu: Two names for one notebook

Ramu writes `b = a`. He thinks he has made a copy of Somu's notebook. Somu: no, you have only written a second name on the same notebook's cover. When Ramu scribbles in it through the name `b`, the notebook seen through `a` changes too. To make a real (shallow) copy use `a[:]` or `a.copy()`; for nested structures use `copy.deepcopy`.

```
a = [1,2,3]; b = a; b.append(4); print(a) # alias
c = a[:]; c.append(5); print(a, c) # shallow copy
n = [[1,2],[3,4]]; sh = n[:]; sh[0].append(99); print(n) # inner list shared!
g = [[0]*2]*2; g[0][0] = 5; print(g) # both rows are the SAME list
a = [1,2]; b = a; b += [3]; print(a) # += changes in place
a = [1,2]; b = a; b = b + [3]; print(a) # + makes a new list
```

```
[1, 2, 3, 4]
[1, 2, 3, 4] [1, 2, 3, 4, 5]
[[1, 2, 99], [3, 4]]
[[5, 0], [5, 0]]
[1, 2, 3]
[1, 2]
```

1.5 Control flow, comprehensions, built-ins

Key facts

- `for x in iterable:` and `while cond:`. `break` leaves the loop; `continue` skips to the next iteration.
- **Loop `else`:** the `else` block of a loop runs only if the loop finished *without* `break`.

- List comprehension: `[expr for x in it if cond]`; dict comprehension `{k:v for ...}`.
- `sorted(it, key=f)` returns a new sorted list (stable); `list.sort()` sorts in place and returns `None`. Sorting uses Timsort: $\mathcal{O}(n \log n)$.
- `map`, `filter`, `zip`, `enumerate`, `sum`, `min`, `max`, `len`, `any`, `all`. `zip` stops at the shortest input.
- Sets: `&` intersection, `|` union, `-` difference, `^` symmetric difference.
- `d.get(k, default)` avoids `KeyError`. Iterating a dict gives its keys.

```
print([i*i for i in range(6) if i%2==0], {i:i*i for i in range(4)})
for i in range(3):
    if i == 5: break
else:
    print("loop finished without break")
words = ["pear","fig","banana"]
print(sorted(words,key=len), sorted(words), words)
print(list(map(lambda t:t*2,[1,2,3])), list(filter(lambda t:t%2,[1,2,3,4])), list(zip("ab
    ",[1,2,3])))
print({1,2,3}&{2,3,4}, {1,2}|{2,5}, {1,2,3}-{2})
```

```
[0, 4, 16] {0: 0, 1: 1, 2: 4, 3: 9}
loop finished without break
['fig', 'pear', 'banana'] ['banana', 'fig', 'pear'] ['pear', 'fig', 'banana']
[2, 4, 6] [1, 3] [('a', 1), ('b', 2)]
{2, 3} {1, 2, 5} {1, 3}
```

1.6 Functions, scope and recursion

Ramu & Somu: The shared shopping bag

Somu defines a helper function `f(x, L=[])` that adds `x` to a list and returns the list. Ramu calls `f(1)` then `f(2)` and is surprised to get `[1]` and then `[1, 2]`. Somu: the default list is created *once*, when the function is defined, and then reused in every call. It is like one shopping bag hanging on the door that every visitor puts things into. The safe pattern is `def f(x, L=None): L = [] if L is None else L`.

```
def f(x, L=[]):
    L.append(x); return L
print(f(1), f(2), f(3,[]))

x = 5
def h():
    x = 10 # local x
h(); print(x)
def cnt():
    global x
    x += 1
cnt(); print(x)
```

```
[1, 2] [1, 2] [3]
5
6
```

(The first line prints `[1, 2]` twice because both calls return the *same* list object, which has grown to `[1, 2]` by the time `print` runs.)

Scope, recursion, arguments

- **LEGB** lookup order: Local, Enclosing function, Global, Built-in. Assigning inside a function makes the name local unless declared `global` / `nonlocal`.
- Arguments are passed by *object reference*: mutating a list argument inside a function changes the caller's list; rebinding the name does not.
- Default arguments are evaluated once at definition time.
- `*args` collects extra positional arguments into a tuple; `**kwargs` collects keyword arguments into a dict.
- Recursion needs a base case; CPython's default recursion limit is about 1000 frames. A function with no `return` returns `None`.

1.7 Classes and exceptions

```
class C:
    cv = [] # class variable: shared by ALL instances
    def __init__(self):
        self.iv = [] # instance variable: one per object
a1 = C(); a2 = C()
a1.cv.append(1); a1.iv.append(1)
print(a2.cv, a2.iv)

def tryit():
    try:
        return "try"
    finally:
        print("finally runs")
print(tryit())
```

```
[1] []
finally runs
try
```

Key facts

- `self` is the object the method is called on; `__init__` is the constructor.
- Order in `try/except/else/finally`: `else` runs if no exception; `finally` *always* runs, even after a `return` in `try`.
- Common exceptions: `IndexError`, `KeyError`, `ValueError`, `TypeError`, `ZeroDivisionError`.

1.8 Strings: useful methods and formatting

Key facts

- Strings are immutable, so every method returns a *new* string: `upper()`, `lower()`, `strip()`, `replace(a,b[,count])`, `split(sep)`, `sep.join(list)`, `startswith()`, `endswith()`.
- `s.find(x)` returns the first index or -1; `s.index(x)` raises `ValueError` if absent; `s.count(x)` counts non-overlapping occurrences.
- `"a b c".split()` (no argument) splits on any whitespace and drops empty pieces: `['a','b','c']`; `"a,b,c".split(",")` gives `['a','b','c']`.
- Formatting: `f"{3.14159:.2f}"` gives `3.14`; `"{:>5}".format(7)` gives `"  7"` (width 5, right aligned); `"%d %s" % (5,"x")` gives `5 x`.

- `len(s)`, `s*2` (repeat), `s1+s2` (concatenate), `"a" in s`, `s[::-1]` (reverse). Comparing strings is lexicographic: `"a"<"b"` is `True`, `"Z"<"a"` is `True` (uppercase letters come first in ASCII).

1.9 Output-tracing workshop (solved step by step)

Ramu & Somu: Be the computer

Somu tells Ramu: "In a GATE Python question you are the computer. Keep a small table of variable values, and execute one line at a time." Try each problem before reading the answer.

Problem 1: recursion.

```
def f(n):
    if n <= 1: return 1
    return f(n-1) + f(n-2)
print(f(5))
```

```
8
```

Base cases $f(0) = f(1) = 1$. Then $f(2) = 2$, $f(3) = 3$, $f(4) = 5$, $f(5) = 8$ (Fibonacci). The number of calls grows exponentially: this naive recursion is $\Theta(\varphi^n)$.

Problem 2: mutating through a function.

```
def g(x):
    x.append(len(x)); return x
a = [1]; b = g(a); b = g(b); print(a, b)
```

```
[1, 1, 2] [1, 1, 2]
```

`g` mutates and returns the *same* list. First call: `len(x)=1`, so `[1,1]`. Second call: `len(x)=2`, so `[1,1,2]`. Since `a`, `b` are the same object both print the same.

Problem 3: `nonlocal`.

```
x = 10
def outer():
    x = 20
    def inner():
        nonlocal x
        x += 1; return x
    return inner()
print(outer(), x)
```

```
21 10
```

`nonlocal x` refers to `outer`'s `x` (20), which becomes 21. The global `x` is untouched.

Problem 4: nested comprehension and slicing.

```
print([i*j for i in range(3) for j in range(i)])
print(sum([i for i in range(10) if i%3==0]))
s = "abcabc"
print(s.count("a"), s.find("c"), s.replace("a","X",1), s[::-1][:2])
t = (1,2,3); a, *b = t; print(a, b)
```

```
[0, 0, 2]
18
2 2 Xbcabc cb
1 [2, 3]
```

The pairs (i, j) with $j < i$ are $(1,0), (2,0), (2,1)$ giving products $0, 0, 2$. Multiples of 3 below 10 sum to $0 + 3 + 6 + 9 = 18$. In the unpacking `a, *b = t` the starred name collects the rest as a *list*.

Problem 5: the late-binding trap in closures.

```
fs = [lambda: i*i for i in range(3)]
print([f() for f in fs])
gs = [lambda i=i: i*i for i in range(3)]
print([g() for g in gs])
```

```
[4, 4, 4]
[0, 1, 4]
```

A lambda looks up `i` when it is *called*, by which time the loop has finished with `i = 2`, so every function returns 4. Binding the value as a default argument (`i=i`) captures it at creation time.

Problem 6: + versus += on lists.

```
x = [1, 2, 3]; y = x
y = y + [4] # builds a new list, y now points to it
x += [5] # extends the original list in place
print(x, y)
```

```
[1, 2, 3, 5] [1, 2, 3, 4]
```

Problem 7: loop cost by counting.

```
def cnt(n):
    c = 0
    for i in range(1, n+1):
        j = 1
        while j <= i:
            j *= 2; c += 1
    return c
print(cnt(8))
```

```
21
```

For each i the inner loop runs $\lfloor \log_2 i \rfloor + 1$ times: for $i = 1..8$ that is $1, 2, 2, 3, 3, 3, 3, 4$, total 21. In general $\sum_{i \le n} \log i = \Theta(n \log n)$.

Problem 8: dictionary, get, enumerate, built-ins.

```
d = {'x': 1}; d['y'] = d.get('y', 0) + 5; print(d, len(d))
print(list(enumerate("ab", start=1)))
print(any([]), all([]), max("abc"), min([3,1,2]), 7//2*2, 2**-1)
```

```
{'x': 1, 'y': 5} 2
[(1, 'a'), (2, 'b')]
False True c 1 6 0.5
```

`any([])` is `False` (no true item); `all([])` is `True` (vacuously); `2**-1` is the float 0.5.

Common GATE traps

(1) `-7//2` is -4, not -3. (2) `[[0]*2]*2` makes two references to the same row. (3) Mutable default arguments. (4) `list.sort()` returns `None`, so `x = x.sort()` destroys your list. (5) Class variables are shared by every instance. (6) `round(2.5)` is 2. (7) Slicing never raises errors for out-of-range bounds, but indexing does.

2 Asymptotic Analysis

Ramu & Somu: Finding a name in the phone book

Ramu needs the number of "Sharma" in a paper phone book of n pages. Method 1: read page by page from the start: up to n pages. Method 2: open in the middle, see whether "Sharma" is before or after, discard half, and repeat: about $\log_2 n$ pages. For $n = 1{,}000{,}000$ pages Method 1 may need a million looks, Method 2 about 20. We do not care about exact seconds (they depend on the machine) but about *how the work grows with* n. That is asymptotic analysis.

Definitions

- $f(n) = \mathcal{O}(g(n))$: there exist c, n_0 with $f(n) \le c\,g(n)$ for all $n \ge n_0$ (upper bound).
- $f(n) = \Omega(g(n))$: $f(n) \ge c\,g(n)$ eventually (lower bound).
- $f(n) = \Theta(g(n))$: both, i.e. same growth rate.
- Growth order: $1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < n! < n^n$.
- Drop constants and lower-order terms: $3n^2 + 10n + 7 = \Theta(n^2)$. $\log_a n = \Theta(\log_b n)$ for any bases.
- Sum of first n integers $= \frac{n(n+1)}{2} = \Theta(n^2)$; $\sum 2^i = 2^{k+1} - 1$; $\sum \frac{1}{i} = \Theta(\log n)$.
- Best/worst/average case describe the *input*, whereas $\mathcal{O}, \Omega, \Theta$ describe the *bound*: do not mix them up.

2.1 Counting loop iterations

Code pattern	Cost
`for i in range(n):` constant work	$\Theta(n)$
`for i in range(n): for j in range(n):`	$\Theta(n^2)$
`for i in range(n): for j in range(i):`	$\frac{n(n-1)}{2} = \Theta(n^2)$ (45 steps for $n = 10$)
`while j<n: j*=2` (or `j//=2`)	$\Theta(\log n)$
`for i in range(n):` then `while j<n: j*=2`	$\Theta(n \log n)$ (64 steps for $n = 16$)
`for i in 1..n: j=i; while j<=n: j+=i`	$\sum n/i = \Theta(n \log n)$ (50 for $n = 16$)
`while n>1: n=n//2`	$\Theta(\log n)$
Recursion `f(n)=f(n-1)+f(n-2)` (naive Fibonacci)	$\Theta(\varphi^n) \approx \Theta(1.618^n)$, exponential
Recursion with one call on $n/2$ plus constant	$\Theta(\log n)$

Common GATE traps

$\mathcal{O}(n)$ code inside a loop of n iterations gives $\mathcal{O}(n^2)$, but a loop whose counter *doubles* gives only $\log n$ iterations. Also note that `x in some_list` is $\mathcal{O}(n)$ (a hidden loop!), whereas `x in some_set` is expected $\mathcal{O}(1)$.

3 Core Data Structures

Ramu & Somu: A canteen and a pile of plates

In the college canteen Somu notices two things. The pile of plates: you always take the *top* plate and put washed plates on the *top* (Last In, First Out). The queue at the counter: the first person to arrive is served first (First In, First Out). Ramu: "So the way we are allowed to add and remove decides the data structure." Exactly.

3.1 Overview of operations

Structure	Access	Search	Insert	Delete	Notes
Array / Python list	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	append at end amortised $\mathcal{O}(1)$
Singly linked list	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$ at head	$\mathcal{O}(1)$ at head	no random access
Stack (LIFO)	—	—	$\mathcal{O}(1)$	$\mathcal{O}(1)$	push/pop at top
Queue (FIFO)	—	—	$\mathcal{O}(1)$	$\mathcal{O}(1)$	enqueue rear, dequeue front
Hash table	—	$\mathcal{O}(1)$ avg	$\mathcal{O}(1)$ avg	$\mathcal{O}(1)$ avg	worst $\mathcal{O}(n)$
BST (unbalanced)	—	$\mathcal{O}(h)$	$\mathcal{O}(h)$	$\mathcal{O}(h)$	h up to $n-1$
AVL / balanced BST	—	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	height $\leq 1.44 \log_2 n$
Binary heap	min/max $\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	build heap $\mathcal{O}(n)$

3.2 Stack, queue, linked list

Key facts

- **Stack:** `push`, `pop`, `peek`. Python: a list with `append()` and `pop()`. Uses: function calls (the call stack), undo, balanced brackets, DFS, postfix evaluation.
- **Queue:** Python: `collections.deque` with `append()` and `popleft()` (both $\mathcal{O}(1)$). Using `list.pop(0)` is $\mathcal{O}(n)$. Uses: BFS, scheduling. A **deque** allows insert/delete at both ends.
- **Circular array queue** of size n: `rear=(rear+1)%n`. With only front/rear indices it can hold $n-1$ items (to tell full from empty); with a count variable it can hold n.
- **Linked list:** nodes hold `data` and `next`. Insert/delete at head $\mathcal{O}(1)$; at a known node $\mathcal{O}(1)$; search $\mathcal{O}(n)$. Reversing in place $\mathcal{O}(n)$ time, $\mathcal{O}(1)$ space. Find the middle or detect a cycle with slow/fast pointers (Floyd).
- **Postfix evaluation:** scan tokens; push numbers; for an operator pop two operands y then x and push x op y. Example: `2 3 4 * +` gives $2 + (3 \cdot 4) = 14$.

```
from collections import deque
st = []; st.append(1); st.append(2); print(st.pop(), st) # LIFO
q = deque([1,2,3]); q.append(4); print(q.popleft(), list(q)) # FIFO
```

```
2 [1]
1 [2, 3, 4]
```

3.3 Linked list operations in code

Ramu & Somu: A train of coaches

Somu compares a linked list to a train: each coach (**node**) holds some passengers (`data`) and a coupling to the next coach (`next`). You can attach a new coach at the front in one move. But to reach the 5th coach you must walk through coaches 1 to 4: there is no random access.

```
class LNode:
    def __init__(s, d, n=None): s.data = d; s.next = n

class LL:
    def __init__(s): s.head = None
    def push_front(s, d): s.head = LNode(d, s.head) # O(1)
    def push_back(s, d): # O(n)
        n = LNode(d)
        if not s.head: s.head = n; return
        c = s.head
        while c.next: c = c.next
        c.next = n
    def delete(s, d): # O(n)
        if not s.head: return
        if s.head.data == d: s.head = s.head.next; return
        c = s.head
        while c.next and c.next.data != d: c = c.next
        if c.next: c.next = c.next.next
    def reverse(s): # O(n), O(1) space
        prev = None; c = s.head
        while c:
            nxt = c.next; c.next = prev; prev = c; c = nxt
        s.head = prev
    def middle(s): # slow/fast pointers
        slow = fast = s.head
        while fast and fast.next:
            slow = slow.next; fast = fast.next.next
        return slow.data
    def to_list(s):
        out = []; c = s.head
        while c: out.append(c.data); c = c.next
        return out
```

Running it: build $10, 20, 30$ with `push_back`, then `push_front(5)`: the list is $[5, 10, 20, 30]$. After `delete(20)`: $[5, 10, 30]$. After pushing $40, 50$ at the back: $[5, 10, 30, 40, 50]$ with `middle()` $= 30$. After `reverse()`: $[50, 40, 30, 10, 5]$.

Cycle detection (Floyd): move a slow pointer by 1 and a fast pointer by 2; if they ever meet there is a cycle; if the fast pointer reaches `None` there is none. For nodes $1 \to 2 \to 3$ it returns `False`; after pointing node 3 back to node 2 it returns `True`. Time $\Theta(n)$, space $\Theta(1)$.

3.4 Stack applications

Ramu & Somu: Brackets and brackets

Ramu writes the expression `{[()]}` and Somu checks it using a stack of "open" brackets: push each opening bracket; for each closing bracket the top of the stack must be its matching opener; at the end the stack must be empty. For `([)]` the closing `)` meets `[` on top, so it is invalid. For `((` the stack is not empty at the end, so it is invalid.

```
def balanced(s):
    st = []; pairs = {')':'(', ']':'[', '}':'{'}
    for ch in s:
        if ch in '([{': st.append(ch)
        elif ch in ')]}':
            if not st or st.pop() != pairs[ch]: return False
    return not st
print(balanced("{[()]}"), balanced("([)]"), balanced("(("))
```

```
True False False
```

Infix to postfix (operator-precedence rules). Scan left to right. Operand: output it. `(`: push. `)`: pop to output until `(`. Operator: while the stack top is an operator of higher precedence, or equal precedence and the operator is left-associative, pop it to the output; then push the new operator. At the end pop everything. Precedence: `^` (highest, *right*-associative) > `*` `/` > `+` `-`.

```
prec = {'+':1, '-':1, '*':2, '/':2, '^':3}
def to_postfix(e):
    out = []; st = []
    for ch in e:
        if ch.isalnum(): out.append(ch)
        elif ch == '(': st.append(ch)
        elif ch == ')':
            while st[-1] != '(': out.append(st.pop())
            st.pop()
        else:
            while st and st[-1] != '(' and (prec[st[-1]] > prec[ch] or
                        (prec[st[-1]] == prec[ch] and ch != '^')):
                out.append(st.pop())
            st.append(ch)
    while st: out.append(st.pop())
    return ''.join(out)
print(to_postfix("A+B*C-D"), to_postfix("(A+B)*(C-D)"),
      to_postfix("A^B^C"), to_postfix("A-B-C"))
```

```
ABC*+D- AB+CD-* ABC^^ AB-C-
```

Trace for `A+B*C-D`: `A`: out `A`. `+`: push. `B`: out `AB`. `*`: higher than `+`, so push. `C`: out `ABC`. `-`: pop `*` (higher) then pop `+` (equal, left-assoc): out `ABC*+`; push `-`. `D`: out `ABC*+D`. End: pop `-`. Result `ABC*+D-`. Notice $A\hat{B}\hat{C} \to AB\hat{C}$ (right-assoc) but $A-B-C \to AB-C-$ (left-assoc).

3.5 Queue implementations

```
class TwoStackQ: # queue built from two stacks
    def __init__(s): s.i = []; s.o = []
    def enq(s, x): s.i.append(x)
    def deq(s):
        if not s.o: # move only when output stack is empty
            while s.i: s.o.append(s.i.pop())
        return s.o.pop()
q = TwoStackQ(); [q.enq(x) for x in (1,2,3)]
print(q.deq(), q.deq()); q.enq(4); print(q.deq(), q.deq())

class CQ: # circular queue with a count variable
    def __init__(s, n): s.a = [None]*n; s.f = 0; s.cnt = 0; s.n = n
    def enq(s, x):
        if s.cnt == s.n: return False
        s.a[(s.f + s.cnt) % s.n] = x; s.cnt += 1; return True
```

```
    def deq(s):
        x = s.a[s.f]; s.f = (s.f + 1) % s.n; s.cnt -= 1; return x
c = CQ(3); print(c.enq(1), c.enq(2), c.enq(3), c.enq(4))
print(c.deq()); print(c.enq(9), c.a)
```

```
1 2
3 4
True True True False
1
True [9, 2, 3]
```

The two-stack queue gives FIFO order (reversing a reversed order); each element moves at most twice, so every operation is *amortised* $\mathcal{O}(1)$. The circular queue wraps around: after one `deq`, the freed slot 0 is reused by the new element 9.

3.6 Hash tables

Ramu & Somu: Contact list by first digit

Somu stores 100 phone contacts in 10 drawers, using the *last digit* of the number to choose a drawer: that rule is the **hash function**. Looking up a contact means opening one drawer, not all 10. When two numbers end with the same digit they **collide** and must share. Either keep a small list inside each drawer (**chaining**) or put the item in the next free drawer (**open addressing**).

Key facts

- Load factor $\alpha = n/m$ (n keys, m slots).
- **Chaining:** expected cost of a search $\Theta(1+\alpha)$; table can have $\alpha > 1$. Worst case all keys in one chain: $\Theta(n)$.
- **Open addressing** ($\alpha \le 1$): linear probing $h(k,i) = (h(k)+i) \bmod m$ causes *primary clustering*; quadratic probing causes secondary clustering; double hashing $h_1(k) + i\,h_2(k)$ spreads best. Under uniform hashing an *unsuccessful* search costs about $\frac{1}{1-\alpha}$ probes.
- Deleting in open addressing needs a "deleted" marker (tombstone), otherwise later searches stop too early.
- Python `dict` and `set`: expected $\mathcal{O}(1)$ insert, lookup, delete; keys must be hashable (so list keys are not allowed, tuples of immutables are).

Linear probing

Table size 7, $h(k) = k \bmod 7$, insert $15, 22, 8, 29$ (all hash to slot 1). $15 \to$ slot 1. 22: slot 1 taken, slot 2 free. $8 \to$ slot 3. $29 \to$ slot 4. Result (slots 0..6): `[None, 15, 22, 8, 29, None, None]`. With chaining all four would sit in the chain of slot 1.

3.7 Binary trees and binary search trees

Ramu & Somu: A family tree and a guessing game

Ramu draws a family tree: one person on top, each person has at most two children (left and right). Somu turns it into a guessing game by arranging numbers so that for every node, everything in its left subtree is *smaller* and everything in its right subtree *larger*. To find 35 you start at the root and keep going left or right, never looking at the other side: that is why a BST search is fast, as long as the tree is not shaped like a long stick.

Facts about binary trees

- A binary tree with n nodes has $n-1$ edges and $n+1$ null child pointers.
- Maximum nodes at level i (root level 0): 2^i. Maximum nodes in a tree of height h (edges): $2^{h+1}-1$. Minimum height of n nodes: $\lceil \log_2(n+1) \rceil - 1$. Maximum height: $n-1$.
- For a binary tree: #leaves = #nodes with two children +1. A *full* binary tree with n leaves has $n-1$ internal nodes.
- Number of distinct binary trees (or BSTs) with n nodes/keys: Catalan number $C_n = \frac{1}{n+1}\binom{2n}{n}$: $1, 2, 5, 14, 42, \ldots$ for $n = 1, 2, 3, 4, 5$.
- **Traversals:** inorder (L, root, R), preorder (root, L, R), postorder (L, R, root), level-order (BFS). All are $\Theta(n)$.
- The **inorder traversal of a BST is sorted**. A tree is determined uniquely by inorder + preorder or inorder + postorder; preorder + postorder alone is not enough in general.
- BST search/insert/delete: $\Theta(h)$. Inserting keys in sorted order gives a chain with height $n-1$. Deleting a node with two children: replace by its inorder successor (or predecessor).
- **AVL tree:** balance factor $|h_L - h_R| \le 1$ for every node; rebalancing by rotations (LL, RR, LR, RL); height $\Theta(\log n)$.

```
class Node:
    def __init__(s,k): s.k=k; s.l=None; s.r=None
def ins(r,k):
    if r is None: return Node(k)
    if k < r.k: r.l = ins(r.l,k)
    else: r.r = ins(r.r,k)
    return r
def ino(r): return ino(r.l)+[r.k]+ino(r.r) if r else []
def pre(r): return [r.k]+pre(r.l)+pre(r.r) if r else []
def post(r): return post(r.l)+post(r.r)+[r.k] if r else []
def ht(r): return -1 if r is None else 1+max(ht(r.l),ht(r.r))
root = None
for k in [50,30,70,20,40,60,80,35]: root = ins(root,k)
print(ino(root)); print(pre(root)); print(post(root)); print(ht(root))
```

```
[20, 30, 35, 40, 50, 60, 70, 80]
[50, 30, 20, 40, 35, 70, 60, 80]
[20, 35, 40, 30, 60, 80, 70, 50]
3
```

The BST built is: root 50; left child 30 (children 20 and 40, with 35 as left child of 40); right child 70 (children 60 and 80). Height 3.

3.8 BST operations: search, insert, delete

Ramu & Somu: Removing a name from the guessing tree

Back in Somu's guessing tree, removing a number is more delicate than adding one. If the number is a **leaf**, just cut it. If it has **one child**, let the child take its place. If it has **two children**, replace its value by the smallest value of its right subtree (the **inorder successor**) and then delete that successor node, which has at most one child.

```
def search(r, k):
    while r and r.k != k: r = r.l if k < r.k else r.r
    return r is not None
def minnode(r):
    while r.l: r = r.l
    return r
```

```
def delete(r, k):
    if r is None: return None
    if k < r.k: r.l = delete(r.l, k)
    elif k > r.k: r.r = delete(r.r, k)
    else:
        if r.l is None: return r.r # leaf or only right child
        if r.r is None: return r.l # only left child
        s = minnode(r.r); r.k = s.k # two children: copy successor
        r.r = delete(r.r, s.k)
    return r
```

Starting from the tree built from $50, 30, 70, 20, 40, 60, 80, 35$ (inorder $[20, 30, 35, 40, 50, 60, 70, 80]$):

- `search(root,35)` is `True`; `search(root,36)` is `False`.
- Delete 20 (a leaf): inorder $[30, 35, 40, 50, 60, 70, 80]$.
- Delete 40 (one child, 35): inorder $[30, 35, 50, 60, 70, 80]$, preorder $[50, 30, 35, 70, 60, 80]$.
- Delete 50 (the root, two children): successor is 60; inorder $[30, 35, 60, 70, 80]$, preorder $[60, 30, 35, 70, 80]$.

All three cases keep the BST property, and deletion costs $\Theta(h)$.

3.9 Rebuilding a tree from two traversals

Inorder + preorder

Inorder $= D\,B\,E\,A\,F\,C$ and preorder $= A\,B\,D\,E\,C\,F$. The *first* preorder symbol A is the root. In the inorder sequence everything left of A (D, B, E) is the left subtree and everything right of it (F, C) is the right subtree. Repeat: left part has preorder B, D, E, so root B with left D, right E (from inorder $D\,B\,E$). Right part has preorder C, F and inorder $F\,C$, so root C with left child F. The tree is $A(B(D, E), C(F, -))$ and its postorder is $D\,E\,B\,F\,C\,A$ (verified by code).
With inorder + *postorder* the root is the *last* postorder symbol; the rest of the process is identical.

3.10 AVL trees and rotations

Ramu & Somu: Keeping the tree from leaning

If Ramu inserts $10, 20, 30$ in order into an ordinary BST he gets a stick leaning to the right. An **AVL tree** fixes this on the spot: after each insertion, every node's **balance factor** (height of left subtree − height of right subtree) must be $-1, 0$ or $+1$. If a node becomes ± 2, a **rotation** restores balance.

Case	Where the new key went	Fix
LL	into the left subtree of the left child	one right rotation
RR	into the right subtree of the right child	one left rotation
LR	into the right subtree of the left child	left rotation on the child, then right rotation
RL	into the left subtree of the right child	right rotation on the child, then left rotation

Inserting $10, 20, 30$: node 10 has balance factor -2 (case RR); a left rotation at 10 makes 20 the root with children 10 and 30. Inserting $30, 10, 20$ (case LR) also ends with 20 as the root after two rotations.

AVL facts

- Insertion needs at most one (single or double) rotation; deletion may need $\mathcal{O}(\log n)$ rotations. All operations are $\Theta(\log n)$.
- Minimum number of nodes $N(h)$ in an AVL tree of height h (edges): $N(h) = N(h-1) + N(h-2) + 1$ with $N(0) = 1,\ N(1) = 2$, giving $1, 2, 4, 7, 12, 20, \ldots$ Hence $h \le 1.44 \log_2 n$.

3.11 Binary heap and priority queue

Key facts

- A **complete** binary tree stored in an array (1-indexed): parent of i is $\lfloor i/2 \rfloor$, children are $2i$ and $2i+1$. (0-indexed: parent $\lfloor (i-1)/2 \rfloor$, children $2i+1, 2i+2$.)
- **Max-heap:** every parent $\ge$ its children, so the max is at the root. **Min-heap**: parent $\le$ children.
- Insert: put at the end, *sift up*: $\mathcal{O}(\log n)$. Extract-max: move last to root, *sift down*: $\mathcal{O}(\log n)$. Peek $\mathcal{O}(1)$.
- **Build-heap** from an arbitrary array: $\Theta(n)$ (not $n \log n$). Heap sort: $\Theta(n \log n)$.
- A heap with n nodes has $\lceil n/2 \rceil$ leaves, height $\lfloor \log_2 n \rfloor$. In a max-heap the minimum is among the leaves. Searching an arbitrary key is $\mathcal{O}(n)$.
- Python: `heapq` is a *min*-heap; `heappush`, `heappop`.

Build a max-heap by repeated insertion

Insert $10, 20, 15, 30, 40$. After 10: $[10]$. Insert 20: sifts above 10 giving $[20, 10]$. Insert 15: $[20, 10, 15]$. Insert 30: lands at index 4 under 10, swaps with 10, then with 20: $[30, 20, 15, 10]$. Insert 40: lands under 20, swaps with 20, then with 30: $[40, 30, 15, 10, 20]$.

3.12 Heapify and heap sort in code

Ramu & Somu: Sorting by repeatedly picking the champion

Somu wants to sort 5 players by strength. He arranges them in a heap so the strongest is at the top (the **champion**). He takes the champion out and puts it at the end of the list, rebuilds the heap on the rest, and repeats. After picking all champions, the list is sorted. This is heap sort:

n picks, each costing $\log n$.

```
def sift_down(a, i, n): # a is 0-indexed, max-heap
    while True:
        l, r, m = 2*i+1, 2*i+2, i
        if l < n and a[l] > a[m]: m = l
        if r < n and a[r] > a[m]: m = r
        if m == i: return
        a[i], a[m] = a[m], a[i]; i = m
def build_heap(a): # bottom-up, Theta(n)
    for i in range(len(a)//2 - 1, -1, -1): sift_down(a, i, len(a))
def heap_sort(a):
    a = a[:]; build_heap(a)
    for end in range(len(a)-1, 0, -1):
        a[0], a[end] = a[end], a[0] # champion goes to the end
        sift_down(a, 0, end)
    return a
a = [4, 10, 3, 5, 1]; build_heap(a); print(a)
print(heap_sort([4, 10, 3, 5, 1, 8]))
b = [40, 30, 15, 10, 20]; b[0], b[-1] = b[-1], b[0]; sift_down(b, 0, 4); print(b[:4])
```

```
[10, 5, 3, 4, 1]
[1, 3, 4, 5, 8, 10]
[30, 20, 15, 10]
```

Why build-heap is $\Theta(n)$. Sifting down from a node of height h costs $\mathcal{O}(h)$, and there are at most $\lceil n/2^{h+1} \rceil$ nodes of height h. Total $\sum_{h \geq 0} \frac{n}{2^{h+1}} h = \mathcal{O}(n)$, since $\sum h/2^h = 2$. Most nodes are near the bottom, where sifting is cheap. (Building by n repeated insertions costs $\Theta(n \log n)$.)
Extract-max example: heap $[40, 30, 15, 10, 20]$: swap root with the last element 20, shrink to $[20, 30, 15, 10]$, sift 20 down (swap with its larger child 30): $[30, 20, 15, 10]$.
Heap sort is $\Theta(n \log n)$ in all cases, in place, but not stable.

Common GATE traps

(1) Inserting a sorted sequence into an ordinary BST gives height $n - 1$, not $\log n$. (2) Preorder + postorder cannot uniquely rebuild a general binary tree. (3) Build-heap is $\Theta(n)$. (4) `heapq` is a min-heap. (5) In open addressing, never simply empty a deleted slot. (6) Height can be counted in nodes or in edges: read the question's convention.

4 Searching

Ramu & Somu: Looking up a word in a dictionary

Somu challenges Ramu: find "mango" in a dictionary of 1000 words. Ramu starts at page 1 and reads every word (**linear search**). Somu opens at the middle: the word there is "lemon", so "mango" must be in the second half; he opens the middle of that half, and so on. After at most 10 openings he is done (**binary search**). The trick works only because the dictionary is *sorted*.

Key facts

- **Linear search:** scan until found. Best 1 comparison; worst and average (successful) about n and $\frac{n+1}{2}$; $\mathcal{O}(n)$. Works on unsorted data and on linked lists.
- **Binary search:** needs a *sorted array with random access*. Compare with the middle element, discard half. Recurrence $T(n) = T(n/2) + \Theta(1) \Rightarrow \Theta(\log n)$.
- Worst-case number of comparisons (3-way compare per step) for n elements: $\lfloor \log_2 n \rfloor + 1$. For $n = 1000$: 10.
- Space: iterative version $\mathcal{O}(1)$; recursive $\mathcal{O}(\log n)$ stack.
- Binary search on a *linked list* is not $\mathcal{O}(\log n)$ because finding the middle costs $\mathcal{O}(n)$.
- Python: `x in lst` is linear; `bisect.bisect_left(lst, x)` is $\mathcal{O}(\log n)$ on a sorted list.
- Searching a BST is $\mathcal{O}(h)$; hash table $\mathcal{O}(1)$ expected.

```
def bs(A, t):
    lo, hi, steps = 0, len(A)-1, 0
    while lo <= hi:
        steps += 1
        m = (lo + hi) // 2
        if A[m] == t: return m, steps
        if A[m] < t: lo = m + 1
        else: hi = m - 1
    return -1, steps
S = [2,5,8,12,16,23,38,56,72,91]
print(bs(S,23), bs(S,91), bs(S,7))
```

```
(5, 3) (9, 4) (-1, 3)
```

Reading the output: 23 is at index 5 and found after 3 probes; 91 (last element) needs 4 probes; the missing key 7 is rejected after 3 probes.

Trace of searching 23

$lo = 0, hi = 9$: $m = 4$, $A[4] = 16 < 23$, so $lo = 5$. $m = 7$, $A[7] = 56 > 23$, so $hi = 6$. $m = 5$, $A[5] = 23$: found. Three probes, as in the output.

4.1 Binary search variants and probe counts

Ramu & Somu: First day the price crossed 100

Ramu has a sorted list of daily share prices and wants the *first* index whose price is at least 100. Plain binary search only says "found or not". The fix is the **lower bound**: keep the invariant "the answer lies in $[lo, hi]$" and shrink until $lo = hi$.

```
def lower_bound(a, x): # first index with a[i] >= x
    lo, hi = 0, len(a)
    while lo < hi:
```

```
        mid = (lo + hi) // 2
        if a[mid] < x: lo = mid + 1
        else: hi = mid
    return lo
a = [1, 3, 3, 3, 7, 9]
print(lower_bound(a, 3), lower_bound(a, 4), lower_bound(a, 10))
```

```
1 4 6
```

The first occurrence of 3 is at index 1; the insertion point for 4 is 4; for 10 it is $6 = n$ (past the end). The number of occurrences of x is lower_bound$(x+1)$ − lower_bound(x) for integers.

n	1	3	7	15	100	1000	10^6
worst-case probes $\lfloor \log_2 n \rfloor + 1$	1	2	3	4	7	10	20

A million sorted items need only 20 comparisons: this is what $\Theta(\log n)$ feels like.

Common GATE traps

Binary search requires sorted data. When counting comparisons read whether the question means 2-way or 3-way comparisons per step, and whether the element is present. The worst case is $\lfloor \log_2 n \rfloor + 1$ probes, not $\log_2 n$.

5 Sorting

Ramu & Somu: Arranging a hand of cards

Ramu is dealt 6 playing cards and arranges them in order. He picks up one card at a time and slides it into the right place among the cards already sorted in his hand: that is **insertion sort**. Somu prefers another method: scan all cards, pick the smallest, put it first; scan the rest, pick the smallest, put it second: **selection sort**. A third friend repeatedly swaps neighbouring cards that are in the wrong order until none are: **bubble sort**. All three take about n^2 steps. For a big deck, Somu splits it in two halves, sorts each, and merges them like zipping together two sorted piles: **merge sort**, $n \log n$.

5.1 Summary table

Algorithm	Best	Average	Worst	Space	Stable	Notes
Selection	n^2	n^2	n^2	1	no	$\leq n-1$ swaps; always $\frac{n(n-1)}{2}$ comparisons
Bubble	n^*	n^2	n^2	1	yes	*with early-exit flag
Insertion	n	n^2	n^2	1	yes	swaps = inversions; great for nearly sorted
Merge	$n \log n$	$n \log n$	$n \log n$	n	yes	divide and conquer
Quick	$n \log n$	$n \log n$	n^2	$\log n$	no	worst case: bad pivots (e.g. sorted input)
Heap	$n \log n$	$n \log n$	$n \log n$	1	no	uses a heap
Counting	$n+k$	$n+k$	$n+k$	$n+k$	yes	integers in $[0, k]$; not comparison-based

Definitions and facts

- **Stable:** equal keys keep their original relative order. **In-place:** $\mathcal{O}(1)$ (or $\mathcal{O}(\log n)$) extra space.
- An **inversion** is a pair $i < j$ with $A[i] > A[j]$. Sorted array: 0 inversions. Reversed array: $\frac{n(n-1)}{2}$ inversions. Insertion sort does exactly (#inversions) element shifts.
- **Lower bound:** any comparison-based sort needs $\Omega(n \log n)$ comparisons in the worst case (decision tree has $n!$ leaves, height $\geq \log_2 n! = \Theta(n \log n)$).
- **Merge sort:** $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$ in all cases; merging two sorted lists of sizes p, q costs at most $p + q - 1$ comparisons.
- **Quick sort:** partition around a pivot. If the partition is balanced, $T(n) = 2T(n/2) + \Theta(n)$; if it is $(n-1, 0)$ each time, $T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$. With the first/last element as pivot this happens on an already sorted or reverse sorted array. Randomised pivot gives expected $\Theta(n \log n)$.
- Even a 9 : 1 split everywhere still gives $\Theta(n \log n)$; only extremely unbalanced splits are bad.
- **Selection sort** makes $\frac{n(n-1)}{2}$ comparisons *regardless of input.* **Bubble sort** makes $\frac{n(n-1)}{2}$ comparisons on reversed input (and $n-1$ on sorted input with the early-exit flag).
- **Counting sort** counts occurrences then places items; stable; $\Theta(n+k)$. **Radix sort** sorts digit by digit using a stable sort.

5.2 Insertion sort

```
def insertion(A):
    A = A[:]; shifts = 0
    for i in range(1, len(A)):
        key = A[i]; j = i - 1
        while j >= 0 and A[j] > key:
            A[j+1] = A[j]; j -= 1; shifts += 1
        A[j+1] = key
    return A, shifts
print(insertion([5,2,4,6,1,3]))
```

```
([1, 2, 3, 4, 5, 6], 9)
```

Trace on $[5,2,4,6,1,3]$: insert 2: $[2,5,4,6,1,3]$ (1 shift); insert 4: $[2,4,5,6,1,3]$ (1); insert 6: no shift; insert 1: moves past $2,4,5,6$ (4 shifts) $[1,2,4,5,6,3]$; insert 3: moves past $4,5,6$ (3) $[1,2,3,4,5,6]$. Total $1+1+0+4+3=9$, which equals the number of inversions of the array.

5.3 Merge sort

```
def merge_sort(A):
    if len(A) <= 1: return A
    m = len(A)//2
    L, R = merge_sort(A[:m]), merge_sort(A[m:])
    i = j = 0; out = []
    while i < len(L) and j < len(R):
        if L[i] <= R[j]: out.append(L[i]); i += 1 # <= keeps it stable
        else: out.append(R[j]); j += 1
    return out + L[i:] + R[j:]
print(merge_sort([38,27,43,3,9,82,10]))
```

```
[3, 9, 10, 27, 38, 43, 82]
```

Splitting $[38,27,43,3,9,82,10] \to [38,27,43]\,|\,[3,9,82,10] \to \ldots$ down to single elements, then merging back up. Depth $\lceil \log_2 7 \rceil = 3$ levels, each level does $\leq n$ work.

5.4 Quick sort and partition

```
def partition(A, lo, hi): # Lomuto: pivot = last element
    p = A[hi]; i = lo - 1
    for j in range(lo, hi):
        if A[j] <= p:
            i += 1; A[i], A[j] = A[j], A[i]
    A[i+1], A[hi] = A[hi], A[i+1]
    return i + 1
def quick(A, lo, hi):
    if lo < hi:
        p = partition(A, lo, hi)
        quick(A, lo, p-1); quick(A, p+1, hi)
A = [8,3,7,1,5,9,2,6]
print(partition(A, 0, len(A)-1), A)
```

```
4 [3, 1, 5, 2, 6, 9, 8, 7]
```

The pivot 6 ends at index 4 with everything smaller on its left $(3,1,5,2)$ and everything larger on its right $(9,8,7)$. The pivot is now in its final sorted position; only the two sides still need sorting.

Bubble sort comparison counts

With the early-exit flag: sorted input $[1,2,3,4,5]$ makes 4 comparisons ($\Theta(n)$); reversed input $[5,4,3,2,1]$ makes $10 = \frac{5\cdot4}{2}$ comparisons. Inversions: $[2,4,1,3,5]$ has 3, $[5,4,3,2,1]$ has 10.

5.5 Elementary sorts traced, and a sort-selection guide

Ramu & Somu: Sorting a hand of cards

Selection sort: Somu scans the whole hand, picks the smallest card and puts it first, then scans the rest for the next smallest, and so on. **Bubble sort**: he walks along the hand swapping any neighbouring pair that is out of order; each pass carries the largest card to the end like a bubble rising. **Insertion sort**: he picks up cards one by one and slides each into the correct place among the cards already held.

```
def selection_sort(a):
    a = a[:]
    for i in range(len(a)):
        m = min(range(i, len(a)), key=lambda j: a[j])
        a[i], a[m] = a[m], a[i] # at most n-1 swaps
    return a
def bubble_sort(a):
    a = a[:]; n = len(a)
    for p in range(n-1):
        swapped = False
        for j in range(n-1-p):
            if a[j] > a[j+1]: a[j], a[j+1] = a[j+1], a[j]; swapped = True
        if not swapped: break # already sorted: O(n)
    return a
def counting_sort(a, k): # keys in 0..k, stable
    cnt = [0]*(k+1)
    for x in a: cnt[x] += 1
    for i in range(1, k+1): cnt[i] += cnt[i-1] # prefix sums = end positions
    out = [None]*len(a)
    for x in reversed(a): cnt[x] -= 1; out[cnt[x]] = x
    return out
print(selection_sort([64, 25, 12, 22, 11]))
print(bubble_sort([5, 1, 4, 2, 8]))
print(counting_sort([4, 2, 2, 8, 3, 3, 1], 8))
```

```
[11, 12, 22, 25, 64]
[1, 2, 4, 5, 8]
[1, 2, 2, 3, 3, 4, 8]
```

Selection-sort trace on $[64,25,12,22,11]$: $[11,25,12,22,64] \to [11,12,25,22,64] \to [11,12,22,25,64]$, sorted. It uses $\Theta(n^2)$ comparisons always but only $\le n-1$ swaps.
Bubble-sort pass 1 on $[5,1,4,2,8]$: $[1,5,4,2,8] \to [1,4,5,2,8] \to [1,4,2,5,8] \to [1,4,2,5,8]$: the largest of the first four is now in place.
Counting sort runs in $\Theta(n+k)$; the backward scan makes it *stable*, which is why it can serve as the digit sorter inside radix sort.

Stability by example. Sort pairs $(2,a),(1,b),(2,c),(1,d)$ by number. A stable sort gives $(1,b),(1,d),(2,a),(2,c)$ (equal keys keep input order). Selection sort with long-distance swaps can give $(1,b),(1,d),(2,c),(2,a)$, which is unstable.

Sort	Best	Average	Worst	Notes
Selection	n^2	n^2	n^2	few swaps, not stable
Bubble (flag)	n	n^2	n^2	stable
Insertion	n	n^2	n^2	stable, good for nearly sorted data
Merge	$n \log n$	$n \log n$	$n \log n$	stable, $\Theta(n)$ extra space
Quick	$n \log n$	$n \log n$	n^2	in place, not stable
Heap	$n \log n$	$n \log n$	$n \log n$	in place, not stable
Counting	$n + k$	$n + k$	$n + k$	integer keys only, stable

Quicksort with a random pivot makes about $1.39\, n \log_2 n$ comparisons on average. Any comparison-based sort needs $\Omega(n \log n)$ comparisons in the worst case, because $n!$ outcomes need a decision tree of height $\geq \log_2 n!$.

Common GATE traps

(1) Selection sort is *not* stable (swaps long distances); bubble and insertion are stable; merge is stable; quick and heap are not. (2) Quick sort's *worst* case is $\Theta(n^2)$, and sorted input is the classic bad case for first/last pivot. (3) Merge sort needs $\Theta(n)$ extra space. (4) Insertion sort is best on almost sorted data. (5) Counting sort beats $n \log n$ only because it is not comparison-based.

6 Divide-and-Conquer

Ramu & Somu: Marking 1000 answer sheets

The teacher has 1000 answer sheets to sort by roll number. Alone, it takes forever. So she splits the pile among two assistants, each splits their half between two more, and so on until each person holds one sheet (trivially sorted). Then the sorted piles are merged back together, pair by pair. This is the divide-and-conquer recipe: **divide** the problem into smaller copies, **conquer** them recursively, **combine** the answers. The total time is described by a **recurrence**.

6.1 The recurrence and the Master theorem

Master theorem

For $T(n) = a\,T(n/b) + \Theta(n^d)$ with $a \geq 1$, $b > 1$, $d \geq 0$:

- If $a < b^d$ (work at the top dominates): $T(n) = \Theta(n^d)$.
- If $a = b^d$ (balanced): $T(n) = \Theta(n^d \log n)$.
- If $a > b^d$ (work at the leaves dominates): $T(n) = \Theta(n^{\log_b a})$.

General form $f(n)$: compare $f(n)$ with $n^{\log_b a}$. Extended case: $T(n) = aT(n/b) + \Theta(n^{\log_b a} \log^k n)$ gives $\Theta(n^{\log_b a} \log^{k+1} n)$.
Recursion-tree view: level i has a^i subproblems of size n/b^i; add up the work per level; there are $\log_b n$ levels.
Not covered by Master theorem (subtract-style recurrences): $T(n) = T(n-1) + n \Rightarrow \Theta(n^2)$; $T(n) = T(n-1) + 1 \Rightarrow \Theta(n)$; $T(n) = 2T(n-1) + 1 \Rightarrow \Theta(2^n)$; $T(n) = T(n-1) + T(n-2) \Rightarrow \Theta(\varphi^n)$.

Recurrence	Compare	Answer
$T(n) = T(n/2) + 1$ (binary search)	$a = 1, b^d = 1$	$\Theta(\log n)$
$T(n) = 2T(n/2) + n$ (merge sort)	$a = 2 = b^d$	$\Theta(n \log n)$
$T(n) = 2T(n/2) + 1$ (tree traversal)	$a = 2 > 1$	$\Theta(n)$
$T(n) = T(n/2) + n$	$a = 1 < 2$	$\Theta(n)$
$T(n) = 2T(n/2) + n^2$	$a = 2 < 4$	$\Theta(n^2)$
$T(n) = 4T(n/2) + n$	$a = 4 > 2$	$\Theta(n^2)$
$T(n) = 3T(n/2) + n$ (Karatsuba)	$a = 3 > 2$	$\Theta(n^{\log_2 3}) \approx n^{1.585}$
$T(n) = 7T(n/2) + n^2$ (Strassen)	$a = 7 > 4$	$\Theta(n^{\log_2 7}) \approx n^{2.81}$
$T(n) = 8T(n/2) + n^2$ (naive matrix mult.)	$a = 8 > 4$	$\Theta(n^3)$
$T(n) = 2T(n/2) + n \log n$	extended case ($k = 1$)	$\Theta(n \log^2 n)$
$T(n) = T(n-1) + n$	subtract type	$\Theta(n^2)$
$T(n) = 2T(n-1) + 1$	subtract type	$\Theta(2^n)$

Quick check of merge sort's recurrence by running $T(n) = 2T(n/2) + n$ with $T(1) = 1$: for

$n = 2, 4, 8, 16, 32$ one gets $4, 12, 32, 80, 192$, which equals $n \log_2 n + n$, consistent with $\Theta(n \log n)$.

6.2 Classic divide-and-conquer algorithms

Key facts

- **Binary search:** $T(n) = T(n/2) + 1$, $\Theta(\log n)$.
- **Merge sort:** $T(n) = 2T(n/2) + n$, $\Theta(n \log n)$.
- **Quick sort:** $T(n) = T(k) + T(n-k-1) + n$; balanced $\Theta(n \log n)$, worst $\Theta(n^2)$.
- **Fast exponentiation:** $a^n = (a^{n/2})^2$ (times a if n odd): $T(n) = T(n/2) + 1$, $\Theta(\log n)$ multiplications.
- **Max and min together** by recursion: $\lceil 3n/2 \rceil - 2$ comparisons (versus $2n - 2$ naively).
- **Counting inversions** with a modified merge sort: $\Theta(n \log n)$.
- **Maximum subarray:** $T(n) = 2T(n/2) + n$, $\Theta(n \log n)$ (Kadane's algorithm does it in $\Theta(n)$).
- **Karatsuba** integer multiplication $\Theta(n^{1.585})$; **Strassen** matrix multiplication $\Theta(n^{2.81})$; **closest pair of points** $\Theta(n \log n)$.

```
def power(a, n):
    if n == 0: return 1
    h = power(a, n//2)
    return h*h if n % 2 == 0 else h*h*a
print(power(3, 13))
```

```
1594323
```

Computing 3^{13} needed only about $\log_2 13 \approx 4$ levels of recursion instead of 13 multiplications.

6.3 Solving recurrences by substitution; more divide-and-conquer

Ramu & Somu: Guess, then prove

Ramu doesn't trust the Master theorem blindly. Somu shows the **substitution method**: guess the answer, then prove it by induction. For $T(n) = 2T(n/2) + n$ guess $T(n) \le c\, n \log_2 n$. Assume it for $n/2$: $T(n) \le 2c\frac{n}{2} \log_2 \frac{n}{2} + n = cn \log_2 n - cn + n \le cn \log_2 n$ whenever $c \ge 1$. The induction goes through, so $T(n) = \mathcal{O}(n \log n)$.

```
def count_inv(a): # merge sort that counts inversions
    if len(a) <= 1: return a, 0
    m = len(a)//2
    L, x = count_inv(a[:m]); R, y = count_inv(a[m:])
    out, i, j, z = [], 0, 0, 0
    while i < len(L) and j < len(R):
        if L[i] <= R[j]: out.append(L[i]); i += 1
        else: out.append(R[j]); j += 1; z += len(L) - i # all remaining L beat R[j]
    return out + L[i:] + R[j:], x + y + z
print(count_inv([2, 4, 1, 3, 5])[1], count_inv([5, 4, 3, 2, 1])[1])

def max_subarray(a, lo, hi): # divide and conquer, Theta(n log n)
    if lo == hi: return a[lo]
    mid = (lo + hi)//2
    best = max(max_subarray(a, lo, mid), max_subarray(a, mid+1, hi))
    s, left = 0, float('-inf')
    for i in range(mid, lo-1, -1): s += a[i]; left = max(left, s)
    s, right = 0, float('-inf')
```

```
    for i in range(mid+1, hi+1): s += a[i]; right = max(right, s)
    return max(best, left + right) # best crossing the middle
a = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
print(max_subarray(a, 0, len(a)-1))
```

```
3 10
6
```

Inversions are pairs $i < j$ with $a_i > a_j$. $[2, 4, 1, 3, 5]$ has 3 inversions: $(2, 1), (4, 1), (4, 3)$. A reversed array of n items has $\binom{n}{2}$ ($= 10$ for $n = 5$). Insertion sort does exactly one shift per inversion.

Maximum and minimum together. Pair up the elements: compare each pair ($n/2$ comparisons), compare the winners for the max and the losers for the min ($\frac{n}{2} - 1$ each). Total $\frac{3n}{2} - 2$ comparisons, better than the naive $2n - 2$.

Karatsuba multiplication. Multiplying two n-digit numbers by the school method costs $\Theta(n^2)$. Karatsuba uses 3 half-size products instead of 4: $T(n) = 3T(n/2) + \mathcal{O}(n) = \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$. Example 12×34: $a = 1, b = 2, c = 3, d = 4$; $ac = 3$, $bd = 8$, $(a + b)(c + d) = 21$, middle term $= 21 - 3 - 8 = 10$, result $3 \cdot 100 + 10 \cdot 10 + 8 = 408$.

Common GATE traps

(1) Use the Master theorem only for $T(n) = aT(n/b) + f(n)$ with a constant ratio b; $T(n) = T(n - 1) + n$ is *not* of this form. (2) Compare a with b^d, not a with d. (3) Divide-and-conquer is not always faster: naive Fibonacci recursion is exponential because the subproblems overlap.

7 Introduction to Graph Theory and Basic Graph Algorithms

Ramu & Somu: A town's road map

Somu draws the villages of his region as dots and the roads as lines between them. A dot is a **vertex** and a line is an **edge**. Ramu asks three questions: "Can I get from village A to village F?" (connectivity, **traversal**); "What is the fewest number of roads I must use?" (**BFS**); "What is the shortest total distance in kilometres?" (**shortest path**, **Dijkstra**). Almost all of graph algorithms are these kinds of questions asked cleverly.

7.1 Graph vocabulary and counting facts

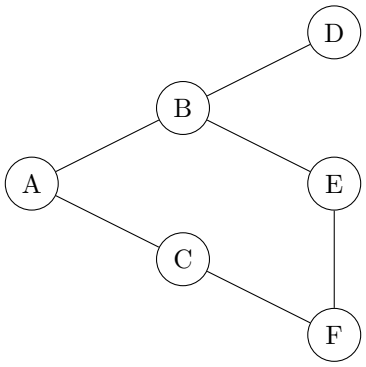
Key facts

- $G = (V, E)$. Undirected: edges are unordered pairs; directed: ordered pairs (arcs). Simple graph: no self-loops, no parallel edges.
- **Degree** of a vertex: number of incident edges. **Handshaking lemma:** $\sum_v \deg(v) = 2|E|$. Hence the number of odd-degree vertices is even.
- Directed graph: $\sum$ in-degree $= \sum$ out-degree $= |E|$.
- Maximum edges in a simple graph on n vertices: $\frac{n(n-1)}{2}$ (undirected, the complete graph K_n), $n(n-1)$ (directed). $K_{m,n}$ has mn edges.
- **Path / cycle / connected:** a connected graph has a path between every pair of vertices. A connected graph on n vertices has at least $n-1$ edges.
- **Tree:** connected and acyclic; has exactly $n-1$ edges; unique path between any two vertices; adding any edge creates exactly one cycle; removing any edge disconnects it. A forest with k components has $n-k$ edges.
- **Bipartite** $\iff$ no odd-length cycle $\iff$ 2-colourable. Trees are bipartite.
- **DAG:** directed acyclic graph; admits a topological ordering.
- **Representations:** adjacency matrix: space $\Theta(V^2)$, edge test $\Theta(1)$, list all neighbours $\Theta(V)$. Adjacency list: space $\Theta(V+E)$, edge test $\Theta(\deg)$, list neighbours $\Theta(\deg)$. Use lists for sparse graphs ($E \ll V^2$).

7.2 Breadth-first search (BFS)

Ramu & Somu: Ripples in a pond

Drop a stone in a pond. The ripple reaches all points at distance 1 first, then distance 2, and so on. BFS explores a graph the same way, using a **queue**: visit the start vertex, then all its neighbours, then all neighbours of those, and so on. So BFS finds the fewest-*edges* path from the start to every vertex.




```
from collections import deque
G = {'A':['B','C'], 'B':['A','D','E'], 'C':['A','F'],
     'D':['B'], 'E':['B','F'], 'F':['C','E']}
def bfs(s):
    seen = {s}; order = []; dist = {s:0}; dq = deque([s])
    while dq:
        u = dq.popleft(); order.append(u)
        for v in G[u]:
            if v not in seen:
                seen.add(v); dist[v] = dist[u] + 1; dq.append(v)
    return order, dist
print(bfs('A'))
```

```
(['A', 'B', 'C', 'D', 'E', 'F'], {'A': 0, 'B': 1, 'C': 1, 'D': 2, 'E': 2, 'F': 2})
```

BFS facts

- Time $\Theta(V + E)$ with adjacency lists ($\Theta(V^2)$ with a matrix). Space $\Theta(V)$.
- Gives shortest paths in **unweighted** graphs. Also used for connected components, testing bipartiteness, level-order tree traversal.
- The BFS order depends on the order neighbours are listed in: always check the question's convention (usually alphabetical or numeric order).

7.3 Depth-first search (DFS)

Ramu & Somu: Exploring a maze

Ramu is in a maze with chalk. He walks down a corridor as far as he can; at a dead end he returns to the last junction with an unexplored path (**backtracking**) and tries that. DFS is just that: go deep first, then back up. It uses a **stack** (or recursion).

```
def dfs(u, seen, order):
    seen.add(u); order.append(u)
    for v in G[u]:
        if v not in seen:
            dfs(v, seen, order)
    return order
print(dfs('A', set(), []))
```

```
['A', 'B', 'D', 'E', 'F', 'C']
```

Read the order: from A go to B; from B go to D (dead end, back to B); then E; from E go to F; from F go to C. Compare with BFS on the same graph: A, B, C, D, E, F.

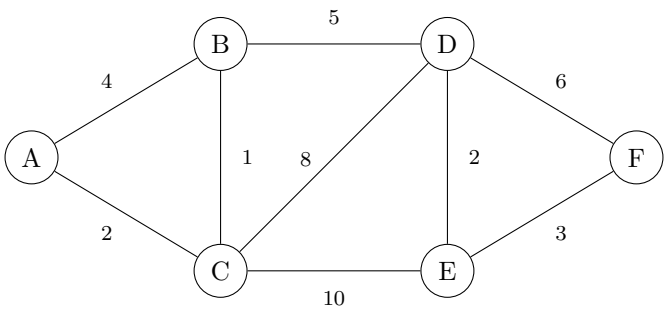
DFS facts

- Time $\Theta(V+E)$ with adjacency lists. Space $\Theta(V)$ (recursion depth up to V).
- Each vertex gets a discovery time $d[v]$ and finish time $f[v]$. Edge types in a directed graph: **tree, back, forward, cross**. In an undirected graph only tree and back edges exist.
- **Cycle detection:** directed graph has a cycle $\iff$ DFS finds a back edge. Undirected: an edge to an already visited vertex that is not the parent.
- **Topological sort** (DAGs): vertices in decreasing finish time of DFS; or Kahn's algorithm: repeatedly remove a vertex of in-degree 0. $\Theta(V+E)$. The order is generally not unique.
- **Connected components** (undirected): run DFS/BFS from each unvisited vertex: number of runs = number of components. Strongly connected components (directed): Kosaraju/-Tarjan, $\Theta(V+E)$.

Topological sort with Kahn's algorithm

DAG with edges $5\to2$, $5\to0$, $4\to0$, $4\to1$, $2\to3$, $3\to1$. In-degrees: 0:2, 1:2, 2:1, 3:1, 4:0, 5:0. Start with the in-degree-0 vertices 4, 5 and repeatedly remove one and decrease its neighbours' in-degrees. One valid output (verified by code) is $4,5,2,0,3,1$. Other valid orders exist, e.g. $5,4,2,3,1,0$.

7.4 Shortest paths



Ramu & Somu: Fastest route, not fewest roads

On Somu's map each road now has a length. The route with the fewest roads may be long in kilometres. **Dijkstra's algorithm**: keep a table of the best known distance to each village (start at 0, others ∞). Repeatedly take the unfinished village with the smallest distance, and see whether going through it improves its neighbours (**relaxation**). Once a village is finished its distance is final, which is safe only because *no road has negative length.*

```
import heapq
W = {'A':[('B',4),('C',2)], 'B':[('A',4),('C',1),('D',5)],
     'C':[('A',2),('B',1),('D',8),('E',10)],
     'D':[('B',5),('C',8),('E',2),('F',6)],
     'E':[('C',10),('D',2),('F',3)], 'F':[('D',6),('E',3)]}
def dijkstra(s):
    dist = {v: float('inf') for v in W}; dist[s] = 0; pq = [(0, s)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]: continue # stale entry
        for v, w in W[u]:
            if d + w < dist[v]: # relaxation
                dist[v] = d + w; heapq.heappush(pq, (dist[v], v))
    return dist
```

```
print(dijkstra('A'))
```

```
{'A': 0, 'B': 3, 'C': 2, 'D': 8, 'E': 10, 'F': 13}
```

Step (finalise)	A	B	C	D	E	F
start	0	∞	∞	∞	∞	∞
A (0)	0	4	2	∞	∞	∞
C (2)	0	3	2	10	12	∞
B (3)	0	3	2	8	12	∞
D (8)	0	3	2	8	10	14
E (10)	0	3	2	8	10	13
F (13)	0	3	2	8	10	13

Shortest route to F has length 13: $A \to C \to B \to D \to E \to F$ $(2+1+5+2+3=13)$.

Shortest path algorithms

Algorithm	Solves	Time	Negative edges?
BFS	single source, unweighted	$\Theta(V+E)$	n/a
Dijkstra	single source, weights ≥ 0	$\Theta(V^2)$ array, $\Theta((V+E)\log V)$ binary heap	**no**
Bellman–Ford	single source	$\Theta(VE)$	yes; detects negative cycles
Floyd–Warshall	all pairs	$\Theta(V^3)$	yes (no negative cycles)
DAG relaxation in topological order	single source on a DAG	$\Theta(V+E)$	yes

Bellman–Ford: relax *all* edges, repeated $V-1$ times; one more pass that still improves something means a negative cycle is reachable. **Floyd–Warshall:** $d_{ij} \leftarrow \min(d_{ij}, d_{ik}+d_{kj})$ for $k = 1..V$ in the outer loop.

Bellman–Ford with a negative edge

Edges $S \to A(4)$, $S \to B(5)$, $A \to C(-3)$, $B \to A(-2)$, $C \to D(2)$, $B \to D(6)$. After $V-1=4$ rounds the distances from S are $A=3$ (via $S \to B \to A$: $5-2$), $B=5$, $C=0$, $D=2$ (verified by code). Dijkstra could be fooled by edges like -2 because it finalises vertices too early.

7.5 Bonus: minimum spanning trees

Key facts

- A **spanning tree** of a connected graph on n vertices has $n-1$ edges. The **MST** is the spanning tree of least total weight.
- **Kruskal:** sort edges by weight, add an edge if it does not create a cycle (union–find): $\Theta(E \log E)$. **Prim:** grow a tree from a vertex, always adding the cheapest edge leaving it: $\Theta(E \log V)$ with a heap.
- If all edge weights are distinct the MST is unique. The cheapest edge of any cut belongs to some MST.

On the weighted graph above, Kruskal picks B–C (1), A–C (2), D–E (2), E–F (3), B–D (5) for total weight 13. It skips A–B (4), which would form a cycle.

7.6 DFS timestamps, edge types and graph tests

Ramu & Somu: Marking time on every door

During DFS Ramu notes a discovery time $d[v]$ when he first enters a room and a finish time $f[v]$ when he leaves it for good. Rooms entered later but left earlier are nested inside rooms entered earlier, like brackets. These timestamps classify every edge of a directed graph:

- **Tree edge**: leads to an undiscovered vertex.
- **Back edge**: leads to an ancestor still on the DFS stack. A directed graph has a cycle *if and only if* DFS finds a back edge.
- **Forward edge**: leads to a descendant already finished.
- **Cross edge**: leads to a vertex in a different, already finished branch.

In an *undirected* graph DFS only yields tree and back edges.

```
def has_cycle_directed(g):
    color = {v: 0 for v in g} # 0 white, 1 grey (on stack), 2 black
    def dfs(u):
        color[u] = 1
        for w in g[u]:
            if color[w] == 1: return True # back edge
            if color[w] == 0 and dfs(w): return True
        color[u] = 2; return False
    return any(color[v] == 0 and dfs(v) for v in g)

def bipartite(g): # BFS 2-colouring, g undirected
    col = {}
    for s in g:
        if s in col: continue
        col[s] = 0; q = [s]
        for u in q:
            for w in g[u]:
                if w not in col: col[w] = 1 - col[u]; q.append(w)
                elif col[w] == col[u]: return False
    return True
print(has_cycle_directed({1:[2], 2:[3], 3:[]}),
      has_cycle_directed({1:[2], 2:[3], 3:[1]}))
print(bipartite({1:[2,4], 2:[1,3], 3:[2,4], 4:[3,1]}),
      bipartite({1:[2,3], 2:[1,3], 3:[1,2]}))
```

```
False True
True False
```

A graph is bipartite exactly when it has *no odd cycle*; the square is bipartite, the triangle is not. The number of connected components equals the number of times the outer loop starts a fresh BFS/DFS.

7.7 Bellman–Ford, Floyd–Warshall and Prim

Ramu & Somu: When roads can pay you

Dijkstra fails if an edge has negative weight, because it assumes a finalised distance can never improve. **Bellman–Ford** makes no such assumption: it relaxes *every* edge, and repeats this $V-1$ times. Any shortest path has at most $V-1$ edges, so after $V-1$ rounds all distances are correct. One more round that still improves something reveals a **negative cycle** reachable from the source.

```
def bellman_ford(n, edges, s):
    d = [float('inf')]*n; d[s] = 0
    for _ in range(n-1):
        for u, v, w in edges:
            if d[u] + w < d[v]: d[v] = d[u] + w
    neg = any(d[u] + w < d[v] for u, v, w in edges)
    return d, neg
print(bellman_ford(4, [(0,1,4),(0,2,5),(1,2,-3),(2,3,2)], 0))
print(bellman_ford(3, [(0,1,1),(1,2,-1),(2,1,-1)], 0)[1])
```

```
([0, 4, 1, 3], False)
True
```

Time $\Theta(VE)$. In the first graph the path $0 \to 1 \to 2$ costs $4-3=1$, beating the direct edge $0 \to 2$ of cost 5.

Floyd–Warshall computes all-pairs shortest paths by dynamic programming over "allowed intermediate vertices":

$$d_{ij}^{(k)} = \min\bigl(d_{ij}^{(k-1)},\; d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\bigr),$$

three nested loops, time $\Theta(V^3)$, space $\Theta(V^2)$. It handles negative edges (but not negative cycles).

Prim grows a single tree from any start vertex, always adding the cheapest edge leaving the tree; with a binary heap it takes $\mathcal{O}(E \log V)$. Kruskal and Prim always give the same total weight (the MST weight is unique even when the tree is not).

Common GATE traps

(1) BFS/DFS visit order depends on adjacency order. (2) Dijkstra fails with negative edges; BFS gives shortest paths only when edges are unweighted. (3) A graph has a topological order iff it is a DAG. (4) $\sum \deg = 2|E|$ (not $|E|$). (5) A tree on n vertices has $n-1$ edges, always. (6) Adjacency matrix BFS costs $\Theta(V^2)$, not $\Theta(V+E)$.

8 Solved GATE-style Problems

Ramu & Somu: Practice with a stopwatch

Ramu has read everything; now Somu hands him exam-style questions. His rule: write the *idea* first (which formula, which structure), then compute. Every answer below was checked by running code or by direct counting.

1. **Recurrence.** Solve $T(n) = 4T(n/2) + n^2$.
   Solution. $a=4,\ b=2$, so $n^{\log_b a} = n^2$, which matches $f(n)=n^2$ (Case 2): $T(n) = \Theta(n^2 \log n)$.

2. **Heap.** Where can the maximum of a min-heap with 15 elements be?

Solution. The maximum cannot have a child that is larger, so it is a leaf. A complete tree on 15 nodes has 8 leaves, so the maximum is among those 8 positions (indices 8 to 15).

3. **Hashing.** A chained hash table has $m = 20$ slots and $n = 50$ keys (simple uniform hashing). Expected keys examined in an unsuccessful search?
Solution. $\alpha = n/m = 2.5$; an unsuccessful search scans a whole chain, expected length $\alpha = 2.5$. (Successful search: about $1 + \alpha/2 = 2.25$.)

4. **Counting trees.** How many structurally distinct binary trees have 4 nodes?
Solution. Catalan number $C_4 = \frac{1}{5}\binom{8}{4} = 14$.

5. **Dijkstra order.** Undirected weighted graph with edges AB=4, AC=2, CB=1, BD=5, CD=8, DE=3, DF=6, EF=1. In what order does Dijkstra from A finalise vertices?
Solution. $A(0)$, $C(2)$, $B(3$ via $C)$, $D(8$ via $B)$, $E(11)$, $F(12)$. Order: A, C, B, D, E, F.

6. **Quicksort worst case.** Comparisons made by quicksort (first element as pivot) on an already sorted array of 10 elements?
Solution. Every partition is maximally unbalanced: $9 + 8 + \cdots + 1 = 45 = \binom{10}{2}$.

7. **Insertion sort.** Reversed array of 6 elements. Number of shifts and key comparisons?
Solution. Shifts = inversions = $\binom{6}{2} = 15$. Each new element is compared with all earlier ones: $1 + 2 + 3 + 4 + 5 = 15$ comparisons.

8. **Forest.** A forest has 8 vertices and 2 trees (components). How many edges?
Solution. A forest has $V - c$ edges: $8 - 2 = 6$.

9. **Postfix.** Evaluate `5 1 2 + 4 * + 3 -`.
Solution. Push $5, 1, 2$; `+`: 3; push 4; `*`: 12; `+`: 17; push 3; `-`: 14. Answer 14.

10. **Python.** What do `a=[1,2,3]; b=a; a=a+[4]` and `a=[1,2,3]; b=a; a+=[4]` leave in `a` and `b`?
Solution. First: `a` is a *new* list $[1, 2, 3, 4]$, `b` stays $[1, 2, 3]$. Second: `+=` extends in place, so both are $[1, 2, 3, 4]$.

11. **Binary search.** Maximum probes to search 100 sorted items?
Solution. $\lfloor \log_2 100 \rfloor + 1 = 6 + 1 = 7$.

12. **Merge.** Maximum comparisons to merge sorted lists of sizes 5 and 3?
Solution. $m + n - 1 = 7$.

13. **Negative edge.** Edges $S \to A = 2$, $S \to B = 3$, $B \to A = -2$. What does Dijkstra compute for A, and what is the truth?
Solution. Dijkstra finalises A at distance 2 first, but the path $S \to B \to A$ costs 1. It outputs 2; the true distance is 1. Use Bellman–Ford.

14. **Full binary tree.** A full binary tree (every node has 0 or 2 children) has 15 nodes. Number of leaves?
Solution. Leaves = $\frac{n+1}{2} = 8$, internal nodes = 7.

9 One-page Cheat Sheet

Topic	Remember
Python arithmetic	`//` floors ($-7//2 = -4$); `%` has divisor's sign; `**` right-assoc.; `round(2.5)=2`
Python traps	alias `b=a`; `[[0]*2]*2`; mutable default; `+=` on lists in place; class variables shared
Growth order	$1 < \log n < \sqrt{n} < n < n\log n < n^2 < 2^n < n!$
Stack / queue / heap	LIFO / FIFO / priority; heap: parent $\lfloor i/2 \rfloor$, children $2i, 2i+1$; build $\Theta(n)$
Hashing	$\alpha = n/m$; chaining $\Theta(1+\alpha)$; open addressing unsuccessful $\approx \frac{1}{1-\alpha}$
Trees	n nodes $\Rightarrow n-1$ edges; leaves = 2-child nodes +1; BST inorder sorted; Catalan counts
Searching	linear $\Theta(n)$; binary $\lfloor \log_2 n \rfloor + 1$ comparisons (sorted array only)
Sorting	insertion/bubble/selection $\Theta(n^2)$; merge $\Theta(n\log n)$ stable; quick worst $\Theta(n^2)$; lower bound $\Omega(n\log n)$
Master theorem	$T = aT(n/b) + n^d$: $a < b^d \to n^d$; $a = b^d \to n^d \log n$; $a > b^d \to n^{\log_b a}$
Graph basics	$\sum \deg = 2\lvert E\rvert$; tree has $n-1$ edges; K_n has $\frac{n(n-1)}{2}$ edges; bipartite iff no odd cycle
BFS / DFS	both $\Theta(V+E)$; BFS = queue, shortest unweighted paths; DFS = stack, cycles, topological sort
Shortest paths	Dijkstra (no negative edges) $\Theta((V+E)\log V)$; Bellman–Ford $\Theta(VE)$; Floyd–Warshall $\Theta(V^3)$

10 Practice Questions (with answers)

Try each on your own first. (All answers were checked by running code where possible.)

1. Output of `print(-7//2, -7%3)`? **Ans:** `-4 2`.
2. `a=[1,2]; b=a; b+=[3]; print(a)`. And with `b=b+[3]`? **Ans:** `[1, 2, 3]`; `[1, 2]`.
3. `f(x,L=[])` appends `x` to `L`. What do the calls `f(1)` then `f(2)` return? **Ans:** `[1]` then `[1, 2]` (shared default list).
4. Worst-case comparisons for binary search on 1000 sorted items? **Ans:** $\lfloor \log_2 1000 \rfloor + 1 = 10$.
5. Solve $T(n) = 3T(n/2) + n$. **Ans:** $a = 3 > 2$: $\Theta(n^{\log_2 3}) \approx n^{1.585}$.
6. Solve $T(n) = 2T(n/2) + n^2$. **Ans:** $a = 2 < 4$: $\Theta(n^2)$.
7. Number of inversions in $[3,1,2]$? And in $[5,4,3,2,1]$? **Ans:** 2; 10.
8. Which input gives quick sort (last-element pivot) its worst case? **Ans:** sorted (or reverse sorted) input: $\Theta(n^2)$.
9. Insertion sort on $[5,2,4,6,1,3]$: how many shifts? **Ans:** 9 (the number of inversions).
10. Max edges in a simple undirected graph on 10 vertices? Edges of a tree on 10 vertices? **Ans:** 45; 9.
11. A graph has 7 edges. Sum of its degrees? **Ans:** 14.
12. BFS from A on the unweighted graph of Section 7 (alphabetical neighbours)? DFS? **Ans:** BFS A, B, C, D, E, F; DFS A, B, D, E, F, C.
13. Shortest distance from A to F in the weighted graph? **Ans:** 13.

14. Table size 7, $h(k) = k \bmod 7$, linear probing, insert $15, 22, 8, 29$: slot of 29? **Ans:** slot 4.
15. Insert $10, 20, 15, 30, 40$ one by one into an empty max-heap: final array? **Ans:** $[40, 30, 15, 10, 20]$.
16. Height (in edges) of the BST obtained by inserting $1, 2, 3, 4, 5$ in this order? **Ans:** 4 (a chain).
17. How many distinct BSTs with 3 keys? **Ans:** Catalan $C_3 = 5$.
18. Time complexity: `for i in range(n):` `j=1` then `while j<n:` `j*=2`. **Ans:** $\Theta(n \log n)$.
19. Evaluate postfix `2 3 4 * +`. **Ans:** 14.
20. Which sorts are stable: selection, bubble, insertion, merge, quick, heap? **Ans:** bubble, insertion, merge.
21. Can Dijkstra be used on a graph with a negative edge? **Ans:** No; use Bellman–Ford.

Practice Set B (with answers)

1. Height of a binary heap with 100 elements (root at height 0)? **Answer:** $\lfloor \log_2 100 \rfloor = 6$.
2. $T(n) = T(n-1) + n$, $T(1) = 1$; value of $T(8)$? **Answer:** 36 $(= 8 \cdot 9/2)$.
3. $T(n) = 2T(n/2) + 1$, $T(1) = 1$; value of $T(8)$? **Answer:** 15 $(= 2n - 1)$.
4. BFS order from 1 for edges $1{\to}2, 1{\to}3, 2{\to}4, 3{\to}4, 3{\to}5, 4{\to}6, 5{\to}6$ (neighbours in listed order)? **Answer:** $1, 2, 3, 4, 5, 6$.
5. DFS order on the same graph? **Answer:** $1, 2, 4, 6, 3, 5$.
6. Linear probing, table size 10, hash $k \bmod 10$, insert $12, 25, 35, 46, 22$. Where does 22 land? **Answer: index** 3 (slot 2 holds 12). Table: slots 2:12, 3:22, 5:25, 6:35, 7:46 (35 moves from 5 to 6, and 46 from 6 to 7).
7. Insert $15, 10, 20, 8, 12, 25, 6$ in a BST. Postorder and height? **Answer:** $6, 8, 12, 10, 25, 20, 15$; **height** 3.
8. Edges in the complete graph K_6? **Answer:** $\binom{6}{2} = 15$.
9. Minimum nodes in an AVL tree of height 4? **Answer:** 12 (sequence $1, 2, 4, 7, 12$).
10. Minimum comparisons needed in the worst case to sort 5 items by any comparison sort? **Answer: at least** $\lceil \log_2 5! \rceil = \lceil \log_2 120 \rceil = 7$.
11. Number of inversions in $[3, 1, 2]$? **Answer:** 2.
12. Time to build a heap from n unsorted items? **Answer:** $\Theta(n)$.
13. Counting sort with n items and keys in $0..k$: time and stability? **Answer:** $\Theta(n + k)$, **stable.**
14. Which graph test uses BFS two-colouring? **Answer: bipartite check.**
15. Cycle in a directed graph is detected by which DFS edge type? **Answer: back edge.**
16. Time of Floyd–Warshall on V vertices? **Answer:** $\Theta(V^3)$.
17. Bellman–Ford time with V vertices and E edges? **Answer:** $\Theta(VE)$.
18. Postfix of $A * (B + C)$? **Answer:** $ABC + *$.
19. `print(7 // 2, -7 // 2, 7 % -2)` gives? **Answer:** `3 -4 -1`.

20. `len(set([1,1,2,3,3]))`? **Answer:** 3**.**

21. `t = (1,[2,3]); t[1].append(4)`: allowed? **Answer: yes**, the tuple still holds the same list object, but the list's contents changed; `t` is `(1, [2, 3, 4])`.

22. Which operation is $\Theta(1)$ on average for a Python dict? **Answer: lookup, insert and delete by key.**

23. Searching a sorted linked list of n nodes: best known time? **Answer:** $\Theta(n)$ (no random access, so no binary search).

24. Worst-case height of a BST on n keys? **Answer:** $n-1$ (a path).

25. Merge sort recurrence and solution? **Answer:** $T(n) = 2T(n/2) + \Theta(n) \Rightarrow \Theta(n \log n)$**.**

How to study this section: Python first (it is mostly output tracing: practise reading code and predicting the output). Then complexity analysis, data structures, searching/sorting, recurrences, and graph algorithms, in that order. Hand-trace each algorithm on a small input at least once, then solve 15–20 GATE-style questions per topic, and keep an error log. Highest-yield for GATE DA: Python output questions, recurrence solving, sorting properties (stability, best/worst case, inversions), BST/heap facts, BFS/DFS orders and Dijkstra tables.